

StenoWav

Technical Reference & Mathematical Formalisation

Version 0.1.0

StenoWav Project

March 2026

Contents

1	Introduction	3
1.1	Notation	3
2	Quantisation Index Modulation (QIM)	3
2.1	Standard QIM ($M = 2$)	3
2.2	Multi-Symbol QIM ($M \geq 2$)	4
2.3	Non-Negative QIM	5
3	Packet Architecture	5
3.1	Packet Structure	5
3.2	Index Selection	5
3.3	Cipher Derivation	5
3.4	Payload Embedding	6
3.5	Stream Framing	6
4	Self-Contained Header	6
5	RMS-Adaptive Placement	7
5.1	Segment Analysis	7
5.2	Power Tier Classification	7
5.3	Pool Construction	7
6	Frequency-Domain Embedding	7
6.1	Analysis	7
6.2	Magnitude Embedding	8
6.3	Synthesis	8
6.4	Frequency-Domain QIM Delta	8
7	Dither	8
7.1	Motivation	8
7.2	Time-Domain Dither	8
7.3	Frequency-Domain Dither	9
7.4	Audibility Analysis	9
8	Quality Metrics	9

9	Security Analysis	10
9.1	Key Space	10
9.2	Extraction Requirements	10
9.3	Statistical Resistance	10
9.4	Per-Packet Cipher	10
10	WAV I/O	10
10.1	Sample Representation	10
11	FFT Implementation	11
12	Complete Pipeline	11
12.1	Encoding	11
12.2	Decoding	12
13	API Reference	12
13.1	Primary Interface	12
13.2	In-Memory Interface	12
13.3	Configuration Parameters	12
14	Capacity Analysis	13
15	Future Directions	13
A	Module Map	13

1 Introduction

StenoWav is an audio steganography framework that embeds arbitrary binary data inside 16-bit PCM WAV audio signals. The core embedding primitive is *Quantisation Index Modulation* (QIM), applied through a packet-based architecture with per-packet cipher obfuscation.

The framework supports two embedding domains:

1. **Time domain** — direct modification of PCM samples.
2. **Frequency domain** — modification of DFT magnitude coefficients, with phase preservation and IDFT reconstruction.

Both paths use RMS-adaptive segment analysis to restrict embedding to signal regions with sufficient energy, and an optional dither pass to raise the statistical noise floor.

This document provides the full mathematical formalisation of every algorithm in the pipeline, along with proofs of key properties (distortion bounds, extraction correctness, security guarantees).

1.1 Notation

Symbol	Meaning
$x \in \mathbb{R}$	A single audio sample (normalised to $[-1, 1]$)
$\Delta \in \mathbb{R}^+$	QIM step size (quantisation interval)
$q \in \mathbb{Z}$	Quantisation index: $q = \text{round}(x/\Delta)$
$M \in \mathbb{N}$	QIM modulus (number of lattice classes)
$s \in \{0, \dots, M-1\}$	Symbol to embed
$b \in \{0, 1\}$	Single bit to embed (special case $M = 2$)
$K \in \{0, \dots, 2^{16} - 1\}$	16-bit cipher key
σ	32-byte master seed
p	Packet number ($p \in \mathbb{N}_0$)
$\mathcal{F}$	Discrete Fourier Transform

2 Quantisation Index Modulation (QIM)

2.1 Standard QIM ($M = 2$)

Definition 2.1 (QIM Lattice). *Given step size $\Delta > 0$, define the quantisation lattice $\Lambda = \{k\Delta : k \in \mathbb{Z}\}$. Partition Λ into two sub-lattices indexed by parity:*

$$\Lambda_b = \{k\Delta : k \equiv b \pmod{2}\}, \quad b \in \{0, 1\}.$$

Definition 2.2 (QIM Embedding). *Given sample x and bit b :*

1. *Compute the quantisation index $q = \text{round}(x/\Delta)$.*
2. *If $q \bmod 2 = b$, set $q^* = q$ (no shift needed).*
3. *If $q \bmod 2 \neq b$, pick the nearest index with correct parity:*

$$q^* = \begin{cases} q + 1 & \text{if } x \geq q\Delta, \\ q - 1 & \text{otherwise.} \end{cases}$$

4. *Output $x' = q^*\Delta$.*

Definition 2.3 (QIM Extraction). *Given watermarked sample x' :*

$$\hat{b} = \text{round}(x'/\Delta) \bmod 2.$$

Theorem 2.4 (Extraction correctness). *For any $x \in \mathbb{R}$, $b \in \{0, 1\}$, and $\Delta > 0$, let $x' = \text{QIM}_\Delta(x, b)$. Then $\text{round}(x'/\Delta) \bmod 2 = b$.*

Proof. By construction, $x' = q^* \Delta$ where $q^* \equiv b \pmod{2}$. Since $\text{round}(q^* \Delta / \Delta) = q^*$, the extracted parity is $q^* \bmod 2 = b$. $\square$

Theorem 2.5 (Distortion bound). *For any embedding $x' = \text{QIM}_\Delta(x, b)$:*

$$|x' - x| \leq \Delta.$$

Proof. In the worst case, x lies exactly at $q\Delta$ with $q \bmod 2 \neq b$, requiring a shift to $q^* = q \pm 1$, giving $|x' - x| = \Delta$. In all other cases the shift is smaller. $\square$

2.2 Multi-Symbol QIM ($M \geq 2$)

Definition 2.6 (Mod- M QIM). *Partition Λ into M sub-lattices: $\Lambda_s = \{k\Delta : k \equiv s \pmod{M}\}$ for $s \in \{0, \dots, M-1\}$.*

Embedding symbol s into sample x :

1. $q = \text{round}(x/\Delta)$.
2. $r = q \bmod M$ (using Euclidean remainder).
3. If $r = s$, output $q\Delta$.
4. Otherwise, compute the forward offset $d = (s - r) \bmod M$:

$$q_\uparrow = q + d, \tag{1}$$

$$q_\downarrow = q + d - M. \tag{2}$$

5. Pick the candidate closer to x :

$$q^* = \text{argmin}_{q' \in \{q_\uparrow, q_\downarrow\}} |q' \Delta - x|.$$

6. Output $x' = q^* \Delta$.

Extraction: $\hat{s} = \text{round}(x'/\Delta) \bmod M$.

Proposition 2.7 (Mod- M distortion bound). $|x' - x| \leq M\Delta/2$ for any mod- M QIM embedding.

Proof. The maximum distance between x and the nearest lattice point of class s is bounded by the half-period of the sub-lattice spacing, which is $M\Delta/2$. $\square$

Remark 2.8. *StenoWav uses $M = 4$ for mask samples (2 bits per sample) and $M = 2$ for data samples (1 bit per sample). The mask modulus $M = 4$ gives a maximum per-sample distortion of 2Δ versus Δ for $M = 2$.*

2.3 Non-Negative QIM

In the frequency domain, embedding operates on magnitude coefficients $|X_k| \geq 0$. Standard QIM can produce negative outputs when $q^* < 0$, but negative magnitudes are physically meaningless and would lose their sign through $\sqrt{\text{Re}^2 + \text{Im}^2}$ on re-extraction.

Definition 2.9 (Non-negative mod- M QIM). *Identical to Definition 2.4, except that $q_\downarrow$ is only considered if $q_\downarrow \geq 0$. If $q_\downarrow < 0$, only $q_\uparrow$ is used. Formally:*

$$q^* = \begin{cases} \operatorname{argmin}_{q' \in \{q_\uparrow, q_\downarrow\}} |q' \Delta - x| & \text{if } q_\downarrow \geq 0, \\ q_\uparrow & \text{otherwise.} \end{cases}$$

Extraction constrains the quantisation index similarly: $\hat{s} = \max(0, \text{round}(x'/\Delta)) \bmod M$.

Proposition 2.10. *Non-negative mod- M QIM preserves extraction correctness for all $x \geq 0$.*

Proof. For $x \geq 0$, $q = \text{round}(x/\Delta) \geq 0$, and the selected $q^* \geq 0$ by construction. Since $q^* \equiv s \pmod{M}$ and $\text{round}(q^* \Delta / \Delta) = q^* \geq 0$, the extraction $\max(0, q^*) \bmod M = s$. $\square$

3 Packet Architecture

3.1 Packet Structure

Each packet embeds 2 bytes (16 bits) of payload using 9 non-contiguous sample positions selected pseudorandomly:

Component	Count	Role
Mask samples	8	Derive per-packet cipher via mod-4 QIM
Data region	1 start index	16 consecutive samples, 1 bit each via mod-2 QIM

Total samples modified per packet: $8 + 16 = 24$.

3.2 Index Selection

Indices are selected via ChaCha20 CSPRNG, seeded from the master seed σ mixed with the packet number p :

$$\sigma_p = \sigma \oplus \underbrace{(0, \dots, 0)}_{24}, p_0, p_1, \dots, p_7,$$

where $p_0, \dots, p_7$ are the little-endian bytes of $p \in \mathbb{N}_0$.

From RNG stream $G(\sigma_p)$, 9 unique indices are drawn via rejection sampling within the eligible pool (see Section 5). The first 8 indices are mask positions; the 9th is the data region start.

3.3 Cipher Derivation

Definition 3.1 (Cipher key). *A 16-bit value $K \in \{0, \dots, 2^{16} - 1\}$ derived from 8 mask samples. Each mask sample i contributes a 2-bit symbol $s_i \in \{0, 1, 2, 3\}$ via mod-4 QIM extraction:*

$$K = \sum_{i=0}^7 s_i \cdot 4^{7-i} = \sum_{i=0}^7 s_i \cdot 2^{2(7-i)}.$$

Equivalently, K is the concatenation of 8 two-bit values $(s_0, s_1, \dots, s_7)$ in big-endian order.

Encoding: A random cipher K is generated from a *separate* ChaCha20 stream (seed byte 0 flipped: $\sigma' = \sigma \oplus (0xFF, 0, \dots, 0)$, then mixed with p). The symbols $s_i = (K \gg 2(7-i)) \& 3$ are embedded into the mask samples via mod-4 QIM.

Decoding: The decoder recomputes mask positions from σ and p , extracts mod-4 residues from each mask sample, and reconstructs K .

3.4 Payload Embedding

Given payload bytes (b_{hi}, b_{lo}) and cipher K :

1. Concatenate into 16-bit word: $P = b_{hi} \cdot 256 + b_{lo}$.
2. Encrypt: $P' = P \oplus K$.
3. Embed P' bit-by-bit into the 16 consecutive samples starting at `data_index`, MSB first, using standard mod-2 QIM.

Extraction:

1. Extract 16 bits from the data region via mod-2 QIM.
2. Reassemble into P' .
3. Decrypt: $P = P' \oplus K$ (XOR is self-inverse).

Proposition 3.2 (XOR cipher involution). $\forall P, K : (P \oplus K) \oplus K = P$.

3.5 Stream Framing

An arbitrary byte stream is framed into packets:

- Each packet carries 2 bytes.
- If the data length is odd, a zero byte is appended.
- Total packets: $N_{\text{data}} = \lceil |\text{data}|/2 \rceil$.
- The decoder uses the original length (from the header) to truncate any padding byte.

4 Self-Contained Header

The first 7 packets embed a 14-byte metadata header, enabling decoding with only the water-marked signal, seed, and configuration.

Bytes	Width	Content
0–3	4	Magic: 0x5354454E (ASCII “STEN”)
4	1	Domain: 0 = Time, 1 = Frequency
5	1	Reserved (0x00)
6–9	4	<code>original_len</code> (u32, big-endian)
10–13	4	<code>num_data_packets</code> (u32, big-endian)

Packet layout:

$$\underbrace{p_0, p_1, \dots, p_6}_{\text{header (7 packets)}} \quad \underbrace{p_7, p_8, \dots, p_{6+N}}_{\text{data (N packets)}}$$

The magic bytes serve as a validation check: if the wrong seed or delta is used, the extracted magic will not match **STEN**, and decoding fails immediately with a descriptive error.

5 RMS-Adaptive Placement

5.1 Segment Analysis

The signal is divided into non-overlapping segments of L samples (default $L = 4096$). For each segment j :

$$\text{RMS}_j = \sqrt{\frac{1}{L_j} \sum_{n \in S_j} x_n^2},$$

where S_j is the set of sample indices in segment j and $L_j = |S_j|$.

5.2 Power Tier Classification

Given thresholds $\theta_H > \theta_L > 0$ (defaults: $\theta_H = 0.10$, $\theta_L = 0.01$):

$$\text{tier}(j) = \begin{cases} \text{HIGH} & \text{if } \text{RMS}_j \geq \theta_H, \\ \text{MID} & \text{if } \theta_L \leq \text{RMS}_j < \theta_H, \\ \text{SKIP} & \text{if } \text{RMS}_j < \theta_L. \end{cases}$$

5.3 Pool Construction

From the classified segments, two pools are constructed:

- **Mask pool:** All individual indices in High or Mid segments.
- **Data pool:** All indices i in High or Mid segments such that $[i, i + 16)$ lies entirely within that segment.

Packet index selection (Section 3) draws from these pools, ensuring no embedding occurs in quiet signal regions where QIM distortion would be perceptible.

Remark 5.1 (Threshold margin). *Thresholds should be set with margin above the QIM delta. QIM modifies samples by at most Δ , shifting the segment RMS by approximately $\Delta/\sqrt{L}$. For $\Delta = 0.005$ and $L = 4096$: $\Delta/\sqrt{L} \approx 7.8 \times 10^{-5}$, which is negligible relative to $\theta_L = 0.01$.*

6 Frequency-Domain Embedding

6.1 Analysis

The signal is partitioned into F non-overlapping frames of W samples (default $W = 1024$, must be a power of 2). Incomplete tail frames are discarded to ensure exact IDFT round-trip.

For each frame f :

$$X_f[k] = \sum_{n=0}^{W-1} x_f[n] e^{-j2\pi kn/W}, \quad k = 0, \dots, W-1, \quad (3)$$

$$|X_f[k]| = \sqrt{\text{Re}(X_f[k])^2 + \text{Im}(X_f[k])^2}, \quad (4)$$

$$\angle X_f[k] = \text{atan2}(\text{Im}(X_f[k]), \text{Re}(X_f[k])). \quad (5)$$

The DFT is computed via a radix-2 Cooley–Tukey FFT in C.

6.2 Magnitude Embedding

Only bins in the range $[k_{\min}, k_{\max})$ are eligible (defaults: $k_{\min} = 10$, $k_{\max} = 400$), excluding DC and very low frequencies which carry perceptually critical content.

The eligible magnitudes from non-Skip frames are flattened into a single array $\mathbf{m} = (m_0, m_1, \dots)$. Mask and data pools are constructed over indices of $\mathbf{m}$ (analogous to Section 5).

Mask samples use non-negative mod-4 QIM (Section 2.3). Data samples use non-negative mod-2 QIM.

6.3 Synthesis

After embedding, the modified magnitudes m'_i are scattered back to their per-frame positions. The time-domain signal is reconstructed:

1. Rebuild the complex spectrum from modified magnitudes and *original* phases: $X'_f[k] = m'_k \cdot e^{j\angle X_f[k]}$ for $k = 0, \dots, W/2$.
2. Enforce conjugate symmetry: $X'_f[W - k] = \overline{X'_f[k]}$ for $k = 1, \dots, W/2 - 1$.
3. Apply IDFT: $x'_f[n] = \frac{1}{W} \sum_{k=0}^{W-1} X'_f[k] e^{j2\pi kn/W}$.

Remark 6.1. *Phase preservation is critical: the decoder must compute the same DFT on the watermarked signal. Since only magnitudes are modified, the decoder's phase angles will differ slightly due to floating-point arithmetic, but the magnitude round-trip is exact up to the DFT's numerical precision.*

6.4 Frequency-Domain QIM Delta

Frequency magnitudes can be much larger than normalised PCM samples. A separate delta Δ_f (default 0.05) controls the frequency-domain step size, independent of the time-domain Δ_t (default 0.005).

The SNR impact in the frequency domain is analogous:

$$\text{SNR}_f = 20 \log_{10} \left(\frac{\text{RMS}(\mathbf{m})}{\Delta_f/2} \right) \text{ dB}.$$

7 Dither

7.1 Motivation

QIM places modified samples on a regular lattice. An attacker performing statistical analysis could detect that some samples sit on a grid while neighbouring samples do not. Dither counters this by adding small perturbations to *all* non-payload samples, making the entire signal appear uniformly noisy.

7.2 Time-Domain Dither

For each sample index n :

1. Generate $u_n \sim \mathcal{U}(-\alpha, +\alpha)$ from a ChaCha20 CSPRNG seeded with $\sigma_{\text{dither}} = \sigma \oplus (0, 0, 0\text{xFF}, 0, \dots)$ (byte 2 flipped for stream independence).
2. If $n \notin \mathcal{P}$ (the set of protected indices): $x_n \leftarrow \text{clamp}(x_n + u_n, -1, 1)$.
3. If $n \in \mathcal{P}$: leave x_n unchanged (but still consume u_n to maintain RNG synchronisation).

Default intensity: $\alpha = 2/32768 \approx 6.1 \times 10^{-5}$ (two least-significant bits of 16-bit PCM).

The protected set $\mathcal{P}$ is the union of all mask indices and data regions [data_index, data_index+16) across all packets.

7.3 Frequency-Domain Dither

For frequency-domain embedding, dither is applied to magnitude bins *before* IDFT synthesis:

1. Generate $u_i \sim \mathcal{U}(-\alpha_f, +\alpha_f)$ per flat-magnitude index.
2. If $i \notin \mathcal{P}_f$: $m_i \leftarrow \max(0, m_i + u_i)$.

The intensity is auto-scaled: $\alpha_f = \alpha \cdot (\Delta_f / \Delta_t)$.

Proposition 7.1 (Dither does not affect extraction). *Since the decoder reads only from indices in $\mathcal{P}$ (or $\mathcal{P}_f$), and dither skips all protected indices, the embedded data is unaffected.*

7.4 Audibility Analysis

The dither noise power per sample is:

$$\sigma_{\text{dither}}^2 = \frac{\alpha^2}{3} \quad \Rightarrow \quad \sigma_{\text{dither}} = \frac{\alpha}{\sqrt{3}}.$$

For the default $\alpha = 6.1 \times 10^{-5}$: $\sigma_{\text{dither}} \approx 3.5 \times 10^{-5}$.

For a typical signal with $\text{RMS}(x) = 0.2$:

$$\text{SNR}_{\text{dither}} = 20 \log_{10} \left(\frac{0.2}{3.5 \times 10^{-5}} \right) \approx 75 \text{ dB}.$$

This is well above the 50 dB empirical threshold measured on real audio (My Own Summer, 16-bit 44.1kHz) and far above the 20 dB threshold generally considered transparent for audio watermarking.

8 Quality Metrics

Given original signal $\mathbf{x}$ and watermarked signal $\mathbf{x}'$, both of length N :

$$\mathbf{d} = \mathbf{x}' - \mathbf{x} \quad (\text{difference signal}) \quad (6)$$

$$\text{SNR} = 20 \log_{10} \left(\frac{\text{RMS}(\mathbf{x})}{\text{RMS}(\mathbf{d})} \right) \text{ dB} \quad \text{where } \text{RMS}(\mathbf{v}) = \sqrt{\frac{1}{N} \sum_n v_n^2} \quad (7)$$

$$\delta_{\max} = \max_n |d_n| \quad (\text{worst-case distortion}) \quad (8)$$

$$\bar{\delta} = \frac{1}{N} \sum_n |d_n| \quad (\text{mean distortion}) \quad (9)$$

$$S_{\text{diff}} = \max_k ||\mathcal{F}(\mathbf{x}')[k]| - |\mathcal{F}(\mathbf{x})[k]|| \quad (\text{max spectral magnitude difference}) \quad (10)$$

Empirical measurements on real audio (My Own Summer, 44.1kHz stereo):

Scenario	SNR (dB)	$\delta_{\max}$	S_{diff}
Embedding only ($\Delta_t = 0.005$)	> 30	< 0.01	< 5.0
Embedding + dither ($\alpha = 2 \text{ LSB}$)	> 30	< 0.01	< 5.0
Pure dither ($\alpha = 2 \text{ LSB}$)	> 50	< 10^{-4}	< 0.5

9 Security Analysis

9.1 Key Space

The master seed σ is 32 bytes (256 bits), giving a key space of 2^{256} . The CSPRNG (ChaCha20) is a well-studied stream cipher with no known practical attacks below brute force.

9.2 Extraction Requirements

An attacker attempting to extract the hidden data needs:

1. **The seed σ :** to locate mask and data sample positions. Without σ , the attacker cannot determine which of the millions of samples carry information.
2. **The QIM delta Δ :** to extract mod-4 residues from mask samples and mod-2 bits from data samples. The wrong Δ yields incorrect quantisation indices and thus garbage output.
3. **Configuration parameters:** domain, RMS thresholds, segment size, frequency bin range. These determine the eligible pools and thus which sample positions the CSPRNG selects from.

9.3 Statistical Resistance

- **Without dither:** Modified samples lie on a QIM lattice (Δ -spaced grid), while unmodified samples retain their original distribution. A statistical test comparing sample-value distributions could detect the lattice pattern.
- **With dither:** All non-payload samples receive random perturbations, obscuring the boundary between modified and unmodified regions. The entire signal exhibits uniform micro-noise.

9.4 Per-Packet Cipher

Each packet generates an independent cipher from a separate ChaCha20 stream (byte 0 of seed flipped). This means:

- Identical payloads in different packets produce different embedded bit patterns.
- Knowledge of one packet's cipher does not reveal others.
- The header magic provides a quick wrong-key check (probability of false positive: $\approx 2^{-32}$).

10 WAV I/O

10.1 Sample Representation

Internal: $x \in [-1.0, 1.0]$ as f64.

Decode (file $\rightarrow$ internal):

$$x = \frac{\text{i16_sample}}{32768.0}.$$

Encode (internal $\rightarrow$ file):

$$\text{i16_sample} = \text{clamp}(\text{round}(x \cdot 32768.0), -32768, 32767).$$

Remark 10.1 (Symmetry constant). *Both directions use 32768.0. An earlier design used 32767.0 for encoding, causing asymmetric quantisation error ($\sim 4 \times 10^{-5}$ vs $\sim 3 \times 10^{-5}$). Using the same constant ensures the round-trip error is symmetric and bounded by $1/(2 \cdot 32768) \approx 1.5 \times 10^{-5}$.*

Theorem 10.2 (QIM survives i16 round-trip). *For $\Delta \gg 1/32768 \approx 3.05 \times 10^{-5}$, QIM-embedded bits survive the $f64 \rightarrow i16 \rightarrow f64$ round-trip.*

Proof sketch. The i16 round-trip introduces at most $\epsilon = 1/(2 \cdot 32768)$ error per sample. For extraction to succeed, the watermarked sample x' must remain in the Voronoi region of its QIM lattice point $q^* \Delta$. The Voronoi radius is $\Delta/2$. If $\epsilon < \Delta/2$, i.e. $\Delta > 2\epsilon \approx 3.05 \times 10^{-5}$, extraction is correct. At $\Delta = 0.005$, the margin is $> 160\times$. $\square$

11 FFT Implementation

The DFT/IDFT is implemented in C using the radix-2 Cooley–Tukey algorithm:

- In-place computation with bit-reversal permutation.
- Input length must be a power of 2.
- Split real/imaginary arrays (`double *re, *im`).
- IDFT via: conjugate $\rightarrow$ FFT $\rightarrow$ conjugate $\rightarrow$ scale by $1/N$.
- Complexity: $O(N \log N)$.

Additional C-level utilities: RMS computation, complex magnitude ($\sqrt{\text{Re}^2 + \text{Im}^2}$), complex phase (`atan2(Im, Re)`).

Rust FFI bindings validate all preconditions (power-of-2 length, matching array lengths) before calling C, returning typed errors via `FftError`.

12 Complete Pipeline

12.1 Encoding

Algorithm 1 StenoWav Encoding Pipeline

Require: WAV file path, data bytes D , seed σ , config C

Ensure: Watermarked WAV file

```

1:  $\mathbf{x} \leftarrow \text{read\_wav}(\text{path})[\text{channel}]$ 
2:  $\text{segments} \leftarrow \text{analyze\_rms}(\mathbf{x}, C)$ 
3:  $(\text{mask\_pool}, \text{data\_pool}) \leftarrow \text{build\_pools}(\text{segments})$ 
4:  $N_D \leftarrow \lceil |D|/2 \rceil$   $\triangleright$  data packets
5:  $H \leftarrow \text{serialize\_header}(\text{domain}, |D|, N_D)$   $\triangleright$  14 bytes
6:  $\text{payload} \leftarrow H \parallel \text{pad}(D)$ 
7: for  $p = 0$  to  $6 + N_D$  do
8:    $\text{idx} \leftarrow \text{select\_indices}(\sigma, p, \text{pools})$ 
9:    $K_p \leftarrow \text{generate\_cipher}(\sigma, p)$ 
10:   Embed  $K_p$  into mask samples via mod-4 QIM
11:   Embed  $\text{payload}[2p : 2p+2] \oplus K_p$  into data region via mod-2 QIM
12: end for
13: if dither enabled then
14:    $\mathcal{P} \leftarrow \text{collect\_protected\_indices}()$ 
15:   Apply dither to  $\mathbf{x}[n]$  for  $n \notin \mathcal{P}$ 
16: end if
17:  $\text{write\_wav}(\text{output}, \mathbf{x})$ 
```

12.2 Decoding

Algorithm 2 StenoWav Decoding Pipeline

Require: Watermarked WAV path, seed σ , config C

Ensure: Extracted data bytes or error

```

1:  $\mathbf{x}' \leftarrow \text{read\_wav}(\text{path})[\text{channel}]$ 
2: Recompute segments and pools (same as encoding)
3: for  $p = 0$  to 6 do                                     ▷ Extract header
4:   Extract cipher  $K_p$  from mask samples
5:   Extract payload  $\oplus K_p$  from data region
6: end for
7: Validate magic = STEN; if wrong, return error
8: Parse  $|D|$  and  $N_D$  from header
9: for  $p = 7$  to  $6 + N_D$  do                               ▷ Extract data
10:  Extract cipher and payload (same as header)
11: end for
12: Truncate to  $|D|$  bytes; return

```

13 API Reference

13.1 Primary Interface

```

Steno::encode(input_path, output_path, data, key, config) -> Result<(),
    StenoError>
Steno::decode(watermarked_path, key, config)                -> Result<Vec<
    u8>, StenoError>

```

13.2 In-Memory Interface

```

Steno::encode_samples(samples, data, key, config) -> Result<(),
    StenoError>
Steno::decode_samples(samples, key, config)        -> Result<Vec<u8>,
    StenoError>

```

13.3 Configuration Parameters

Parameter	Type	Default	Description
domain	enum	Time	Time or Frequency embedding
time_delta	f64	0.005	QIM Δ for PCM samples
freq_delta	f64	0.05	QIM Δ for magnitudes
rms_high	f64	0.10	High power tier threshold
rms_low	f64	0.01	Skip tier threshold
rms_segment_size	usize	4096	Samples per RMS segment
freq_window_size	usize	1024	FFT window (power of 2)
freq_min_bin	usize	10	Lowest eligible frequency bin
freq_max_bin	usize	400	Highest eligible bin (excl.)
dither_enabled	bool	false	Enable noise dither
dither_intensity	f64	$\sim 6.1\text{e-}5$	Max perturbation (~ 2 LSB)
channel	usize	0	Audio channel (0 = left)

14 Capacity Analysis

Each packet carries 2 bytes and uses ~ 24 sample positions. For a message of B bytes:

$$N_{\text{data}} = \lceil B/2 \rceil, \quad (11)$$

$$N_{\text{total}} = 7 + N_{\text{data}}, \quad (12)$$

$$\text{samples used} = 24 \cdot N_{\text{total}}. \quad (13)$$

To avoid packet collisions (overlapping random indices), the eligible pool should be $\gg 24 \cdot N_{\text{total}}$. A conservative guideline:

$$|\text{pool}| \geq 100 \times 24 \cdot N_{\text{total}}.$$

For typical 44.1kHz stereo audio (3 minutes ≈ 7.9 M samples per channel), and assuming 50% of segments are eligible, the pool size is ~ 4 M, supporting ~ 6900 packets ≈ 13.8 KB of hidden data.

15 Future Directions

1. **Variable packet sizes:** Expand beyond 9 samples per packet (e.g., 17 for 32-bit cipher + 32 data bits).
2. **Higher bit density:** Use higher-order QIM ($M > 4$ for masks, $M > 2$ for data) to increase bits per sample.
3. **AWGN robustness optimisation:** Sweep Δ , RMS thresholds, and bit density across thousands of noise samples to find the Pareto frontier of robustness vs. imperceptibility.
4. **Compression robustness:** Test and harden against MP3, AAC, and Opus transcoding.
5. **Streaming / real-time:** Adapt the frame-based architecture for low-latency applications.
6. **Error correction:** Add forward error correction (e.g., Reed–Solomon) for graceful degradation under channel noise.
7. **Multiple channel embedding:** Spread data across stereo channels for increased capacity and redundancy.

A Module Map

File	Responsibility
<code>src/steno.rs</code>	Public API: <code>Steno</code> , <code>StenoConfig</code> , <code>StenoError</code>
<code>src/embed.rs</code>	Dual-domain engine, RMS analysis, header, <code>encode_full/decode_full</code>
<code>src/packet.rs</code>	Packet structure, index selection, encode/decode per packet
<code>src/cipher.rs</code>	Cipher key type, generation, mod-4 QIM embed/extract
<code>src/qim.rs</code>	Core QIM primitives (mod-2)
<code>src/dither.rs</code>	Dither engine, protected index collection, quality metrics
<code>src/fft.rs</code>	Rust FFI bindings to C FFT library
<code>src/wav.rs</code>	16-bit PCM WAV reader/writer
<code>csrc/math_ops.c</code>	C math library: FFT, IFFT, RMS, complex ops
<code>build.rs</code>	Compiles C via <code>cc</code> crate